ReLoop

Second-Hand Product Marketplace

Project Report | 10 October 2026

Item	Detail
Project	Second-hand marketplace managing products, categories, price, condition and sellers
Required stack	MongoDB, Express.js, React, Node.js (MERN)
Build tools used	Google AI Studio (Build mode) for the UI, Stitch for design, Antigravity IDE for the full-stack build
Hosting target	Google Cloud Run (service: reloop-second-hand-product-marketplace, region asia-southeast1)
Current status	UI built; first Cloud Run deployment FAILED (container exited on startup). Fix steps in Section 8.

Basis of this report: it is based on the project specification and prompts used to generate the app, and on the Cloud Run deployment logs. The generated source code itself was not reviewed, so feature descriptions below reflect what was specified and should be verified in the running app (see the checklist in Section 9).

1. Project Overview

ReLoop is a web marketplace where people list, browse and buy pre-owned items. The system supports adding, viewing, updating and managing records for products, categories, prices, conditions and sellers. The interface follows a light, frosted-glass dashboard style with a violet accent, inspired by a luxury car infotainment screen (the Porsche name and logo are deliberately not used).

2. Objectives

- Let sellers create, edit, delete and mark listings as available, reserved or sold.
- Let buyers search, filter, sort, wishlist and contact sellers.
- Give admins control over categories, users and listings.
- Deliver a dynamic, memorable interface (gauges, progress bars, dark mode, animations).
- Keep the data layer swappable so a MongoDB/Express backend can replace the browser-based mock data.

3. Technology Stack

Layer	Technology	Purpose
Database	MongoDB + Mongoose	Stores users, products, categories, wishlists
Backend	Node.js + Express.js	REST API, validation, auth, error handling

Layer	Technology	Purpose
Frontend	React (Vite or AI Studio build), React Router, Tailwind CSS	User interface and routing
Auth	JWT + bcrypt	Login, roles (buyer / seller / admin)
Charts	Recharts + custom SVG gauges	Seller dashboard and Deal Score gauge
AI features	Gemini API	Listing description, price suggestion, photo analysis
Hosting	Google Cloud Run	Container deployment from AI Studio

4. System Architecture

Two build paths were prepared:

- **Path A - Full MERN (Antigravity):** React client on port 5173 talks to an Express API on port 5000, which uses Mongoose to read and write MongoDB. One command (`npm run dev`) starts both.
- **Path B - AI Studio preview:** the browser cannot connect directly to MongoDB, so a service layer (`src/services/api.ts`) is backed by localStorage. Its function signatures mirror the REST endpoints, so replacing it with Axios calls to Express later requires no UI changes.

Request flow (Path A): React page -> Axios (/api) -> Express route -> middleware (auth, validation) -> controller -> Mongoose model -> MongoDB.

5. Data Model

Collection	Fields
User	name, email, password (hashed), role, phone, city, rating, createdAt
Category	name, slug, icon, description
Product	title, description, price, category (ref), condition (New, Like New, Good, Fair, Poor), seller (ref), images[], location, status (available, reserved, sold), negotiable, views, createdAt
Wishlist	user (ref), products[] (refs)

6. Features and Pages

Page	Key features
Home	Deal Score circular gauge, nearby listings card, product status bars, stats counters, trending carousel, condition selector, floating dock navigation
Browse	Live search, category chips, price slider, condition pills, negotiable toggle, sorting, pagination, skeleton loaders
Product details	Image gallery, deal gauge, condition meter, seller card, contact modal, wishlist, related items, view counter
Auth	Login and register with validation, demo credentials, role-based access
Seller dashboard	Stats cards, charts, listing table with status pills, add/edit/delete form, mark sold or reserved
Wishlist	Saved items with empty state

Page	Key features
Admin panel	Manage categories, users and listings
Extras	Dark and light mode, toasts, responsive layout, 404 page

Gemini-powered features: generate listing description, suggest a fair price range, and analyze an uploaded photo to suggest title, category and condition, with graceful fallback on API errors.

7. REST API Summary (Path A)

Area	Endpoints
Auth	POST /api/auth/register, POST /api/auth/login, GET /api/auth/me
Products	GET, POST /api/products; GET, PUT, DELETE /api/products/:id; PATCH /api/products/:id/status. Query: q, category, condition, minPrice, maxPrice, status, sort, page
Categories	GET, POST, PUT, DELETE /api/categories (write operations admin only)
Sellers	GET /api/sellers/:id
Wishlist	GET /api/wishlist; POST, DELETE /api/wishlist/:productId
Stats	GET /api/stats

Security and quality: express-validator input checks, centralized error handler, CORS, ownership checks (only the seller or an admin can edit or delete a listing), hashed passwords and environment variables via .env.

8. Deployment Status and Issue Analysis

Result: The Cloud Run service did not start. Both 'Updating service' and 'Routing traffic' reported: the container failed to start and listen on PORT=3000 within the allowed timeout.

Evidence from logs: the log shows the Node.js v22.23.2 runtime followed by 'Container called exit(1)'. This means the app process crashed during startup, before it could open the port. The 'npm notice' lines (new npm version available) are harmless.

Probable causes (in order of likelihood)

#	Cause	Fix
1	Missing GEMINI_API_KEY; server reads it at startup and throws	Add it under Variables and secrets in Cloud Run; also make code not crash when it is missing
2	Server not bound to process.env.PORT on 0.0.0.0	app.listen(process.env.PORT || 3000, '0.0.0.0')
3	Start script wrong or build step not run	Build in the Docker/build step; 'npm start' must serve the built output
4	Missing dependency or ESM import error	List all packages in package.json; check import paths and extensions

Note: the exact error line sits just above the visible part of the log in the screenshot. Confirming it will identify which cause applies.

Recommended fix procedure

1. Open Cloud Logging and copy the first Error / TypeError line before 'Container called exit(1)'.
2. Run locally: npm install, npm run build, npm start. If it crashes, the same error appears.
3. Apply the matching fix above and set required environment variables in Cloud Run.
4. Redeploy from AI Studio or use 'Edit and deploy new revision' in Cloud Run.
5. Open the service URL and run the checklist in Section 9.

9. Verification Checklist

Area	Test	Result
Startup	Container starts and URL loads	Pending (deployment failing)
Auth	Register, login, role redirect	To verify
Listings	Create, edit, delete, mark sold/reserved	To verify
Browse	Search, filters, sort, pagination	To verify
Wishlist	Add and remove, empty state	To verify
Admin	Category, user, listing management	To verify
AI features	Description and price suggestion work; errors handled	To verify
UI	Dark mode, mobile layout, dock navigation	To verify
Security	No API key in source or repository; .env ignored	To verify

10. Limitations and Risks

- The AI Studio build uses localStorage, so data is per-browser and is not a real MongoDB database. The assignment's MongoDB requirement is met only by the Antigravity (MERN) build.
- A Starter-tier Google Cloud project has lower quotas and may limit scaling.
- Gemini API calls need a valid key, and keys must never be committed to GitHub.
- Unsplash image URLs and local /uploads storage are not production-grade image hosting.

11. Future Improvements

- Buyer-seller chat and notifications.
- Payment integration and order history.
- Cloud image storage (for example Cloud Storage or Cloudinary) and MongoDB Atlas hosting.
- Reviews and ratings, reporting of suspicious listings, and email verification.
- Automated tests (Jest, Supertest, Cypress) and a CI/CD pipeline to GitHub and Cloud Run.

12. Conclusion

ReLoop is specified as a complete second-hand marketplace with full record management, a distinctive dashboard-style interface and AI-assisted listing tools. The remaining blocker is the Cloud Run startup

crash, which is a configuration or startup-code issue (most likely the environment variable or port binding) rather than a design problem. Once the exact error line is confirmed and fixed, the app can be redeployed, verified with the checklist above and published to GitHub.